

```
1.  /**
2.   * @file Controlador para los contactos de emergencia.
3.   * @description Gestiona las operaciones CRUD para los contactos de emergen
4.   * @requires ../models/contactoEmergencia_mongoose
5.   * @requires ../models/usuarios_mongoose
6.   * @requires nodemailer
7.   */
8.  const ContactoEmergencia = require('../models/contactoEmergencia_mongoose')
9.  const User = require('../models/usuarios_mongoose');
10.  const nodemailer = require('nodemailer');
11.
12.  /**
13.   * @function createContacto
14.   * @description Crea un nuevo contacto de emergencia para el usuario autent
15.   * @param {object} req - Objeto de petición de Express.
16.   * @param {object} res - Objeto de respuesta de Express.
17.   * @returns {Promise<void>}
18.   */
19.  exports.createContacto = async (req, res) => {
20.    try {
21.      const { nombre, telefono, email, relacion } = req.body;
22.      const usuario = req.user.id;
23.
24.      const nuevoContacto = new ContactoEmergencia({
25.        usuario,
26.        nombre,
27.        telefono,
28.        email,
29.        relacion
30.      });
31.
32.      await nuevoContacto.save();
33.
34.      await User.findByIdAndUpdate(usuario, { contactoEmergenciaAnadido:
35.
36.      res.status(201).json({ message: 'Contacto de emergencia creado con
37.    } catch (error) {
38.      res.status(500).json({ message: 'Error al crear el contacto de emer
39.    }
40.  };
41.
42.  /**
43.   * @function getContactos
44.   * @description Obtiene todos los contactos de emergencia del usuario auten
45.   * @param {object} req - Objeto de petición de Express.
46.   * @param {object} res - Objeto de respuesta de Express.
47.   * @returns {Promise<void>}
48.   */
49.  exports.getContactos = async (req, res) => {
```

Home

Namespaces

apiConfig

contactoEmergenciaService

diarioService

Classes

ErrorBoundary

Events

SIGINT

unhandledRejection

Global

404_Error_Handler

ACTIVIDADES

API_URL

ASPECTOS_COGNITIVOS

ActionCard

ActivitySelector

App

AuthButtons

AuthLayout

Button

CORS_Configuration

Card

Carousel

Checkbox

CircularProgress

CognitionSelector

DiaryBanner

DiaryEditor

DiaryEntry

EMOCIONES

EmergencyButton

EmergencyModal

EmotionSelector

EmotionStats

Footer

GET_/_

Generic_Error_Handler

Heading

HeroSection

Home

Input

Landing

LandingHero

Login

Logo

MainLayout

Navbar

ProtectedRoute

Register

Seguimiento

Slider

Text

Textarea

accessWithPassword

actualizarEntradaDiario

api

apiFetch

authMiddleware

closeShareModal

compararPassword

```

50.     try {
51.         const contactos = await ContactoEmergencia.find({ usuario: req.user
52.             res.status(200).json(contactos);
53.     } catch (error) {
54.         res.status(500).json({ message: 'Error al obtener los contactos de
55.     }
56. };
57.
58. /**
59.  * @function getContactoById
60.  * @description Obtiene un contacto de emergencia específico por su ID.
61.  * @param {object} req - Objeto de petición de Express.
62.  * @param {object} res - Objeto de respuesta de Express.
63.  * @returns {Promise<void>}
64.  */
65. exports.getContactoById = async (req, res) => {
66.     try {
67.         const contacto = await ContactoEmergencia.findOne({ _id: req.params
68.         if (!contacto) {
69.             return res.status(404).json({ message: 'Contacto de emergencia
70.         }
71.         res.status(200).json(contacto);
72.     } catch (error) {
73.         res.status(500).json({ message: 'Error al obtener el contacto de em
74.     }
75. };
76.
77. /**
78.  * @function updateContacto
79.  * @description Actualiza un contacto de emergencia existente.
80.  * @param {object} req - Objeto de petición de Express.
81.  * @param {object} res - Objeto de respuesta de Express.
82.  * @returns {Promise<void>}
83.  */
84. exports.updateContacto = async (req, res) => {
85.     try {
86.         const { nombre, telefono, email, relacion } = req.body;
87.         const contacto = await ContactoEmergencia.findOneAndUpdate(
88.             { _id: req.params.id, usuario: req.user.id },
89.             { nombre, telefono, email, relacion },
90.             { new: true }
91.         );
92.
93.         if (!contacto) {
94.             return res.status(404).json({ message: 'Contacto de emergencia
95.         }
96.
97.         res.status(200).json({ message: 'Contacto de emergencia actualizado
98.     } catch (error) {
99.         res.status(500).json({ message: 'Error al actualizar el contacto de
100.     }
101. };
102.
103. /**
104.  * @function deleteContacto
105.  * @description Elimina un contacto de emergencia.
106.  * @param {object} req - Objeto de petición de Express.

```

```

componentDidCatch
connectDBAndStartServer
copyShareLink
crearEntradaDiario
create
createContacto
createRegistro
delete
deleteContacto
description
dismiss
duracion
eliminarEntradaDiario
emotion
error
formatDate
formatDateShort
getAll
getById
getContactoById
getContactos
getDerivedStateFromError
getIntensidad
getPreviewText
getProfile
getRegistroByFecha
getRegistroById
getRegistros
getRegistrosByRango
getStepLabel
goToNext
goToPrevious
handleAddActividad
handleAddContact
handleCall
handleCancelAdd
handleCancelEditor
handleChange
handleCreateEntry
handleDeleteEntry
handleEditEntry
handleEmocionToggle
handleInputChange
handleIntensidadChange
handleLogin
handleLogout
handleOverlayClick
handleRegister
handleReload
handleRemoveActividad
handleSendEmail
handleSendSms
handleShareEntry
handleSubmit
handleToggle
handleToggleExpand
handleUpdateEntry
info
intensidad
isMobileDevice
isSelected
loadContacts
loadEntries
loading
loginUser
nextStep
nombre
obtenerEntradaDiarioPorId
obtenerEntradasDiario

```

```

107.  * @param {object} res - Objeto de respuesta de Express.
108.  * @returns {Promise<void>}
109.  */
110.  exports.deleteContacto = async (req, res) => {
111.      try {
112.          const contacto = await ContactoEmergencia.findOneAndDelete({ _id: req.params.id });
113.
114.          if (!contacto) {
115.              return res.status(404).json({ message: 'Contacto de emergencia no encontrado' });
116.          }
117.
118.          const remainingContactos = await ContactoEmergencia.countDocuments({ _id: req.params.id });
119.          if (remainingContactos === 0) {
120.              await User.findByIdAndUpdate(req.user.id, { contactoEmergencia: null }, { upsert: true });
121.          }
122.
123.          res.status(200).json({ message: 'Contacto de emergencia eliminado correctamente' });
124.      } catch (error) {
125.          res.status(500).json({ message: 'Error al eliminar el contacto de emergencia' });
126.      }
127.  };
128.
129.  /**
130.   * @function sendEmergencyEmail
131.   * @description Envía un email de emergencia a un contacto.
132.   * @param {object} req - Objeto de petición de Express.
133.   * @param {object} res - Objeto de respuesta de Express.
134.   * @returns {Promise<void>}
135.   */
136.  exports.sendEmergencyEmail = async (req, res) => {
137.      try {
138.          const { contactoId } = req.params;
139.          const usuario = await User.findById(req.user.id);
140.
141.          // Buscar el contacto de emergencia
142.          const contacto = await ContactoEmergencia.findOne({ _id: contactoId });
143.
144.          if (!contacto) {
145.              return res.status(404).json({ message: 'Contacto de emergencia no encontrado' });
146.          }
147.
148.          if (!contacto.email) {
149.              return res.status(400).json({ message: 'El contacto no tiene email' });
150.          }
151.
152.          // Configurar el transporter de nodemailer
153.          // Usando Gmail como ejemplo - necesitarás configurar las credenciales de Gmail
154.          const transporter = nodemailer.createTransport({
155.              service: 'gmail',
156.              auth: {
157.                  user: process.env.EMAIL_USER,
158.                  pass: process.env.EMAIL_PASS
159.              }
160.          });
161.
162.          // Nombre del usuario que envía

```

```

optionalAuthMiddleware
percentage
port
pre-save
prevStep
promise
registerUser
render
request-interceptor
response-interceptor
sendEmergencyEmail
success
timeAgo
title
togglePasswordField
update
updateContacto
uri
useAuthStore
useToast
validateRequiredEnvVars

```

```

163.     const nombreUsuario = usuario?.nombre || 'Un usuario de MindCare';
164.
165.     // Configurar el mensaje
166.     const mailOptions = {
167.         from: process.env.EMAIL_USER,
168.         to: contacto.email,
169.         subject: 'URGENTE: Solicitud de ayuda de ' + nombreUsuario,
170.         html: `
171.             <div style="font-family: Arial, sans-serif; max-width: 600p
172.                 <div style="background-color: #dc2626; color: white; pa
173.                     <h1 style="margin: 0;"> Mensaje de Emergencia</h1:
174.                 </div>
175.                 <div style="background-color: #f9fafb; padding: 20px; b
176.                     <p style="font-size: 16px; color: #1f2937;">
177.                         Hola <strong>${contacto.nombre}</strong>,
178.                     </p>
179.                     <p style="font-size: 16px; color: #1f2937;">
180.                         <strong>${nombreUsuario}</strong> te ha añadido
181.                     </p>
182.                     <div style="background-color: #fef3c7; border-left:
183.                         <p style="margin: 0; color: #92400e; font-weigh
184.                             Por favor, contacta con esta persona lo ant
185.                         </p>
186.                     </div>
187.                     <p style="font-size: 14px; color: #6b7280;">
188.                         Este mensaje ha sido enviado automáticamente de
189.                     </p>
190.                 </div>
191.             </div>
192.         `;
193.     };
194.
195.     // Enviar el email
196.     await transporter.sendMail(mailOptions);
197.
198.     res.status(200).json({ message: 'Email de emergencia enviado con éx
199. } catch (error) {
200.     console.error('Error al enviar email de emergencia:', error);
201.     res.status(500).json({ message: 'Error al enviar el email de emerge
202. }
203. });
204.

```